



ÆNIX CERTIFICATION FOR COZYSTACK

CCF exam study materials

Seven lessons and a cheat sheet — exactly what the Cozystack Certified Fundamentals exam asks about.

-
- 1 What Cozystack is made of
 - 2 Tenants and access
 - 3 Managed applications catalog
 - 4 Virtual machines
 - 5 Networking
 - 6 Observability and backups
 - 7 How it all actually works
 - 8 Cheat sheet
-

1. What Cozystack is made of

Three layers, four installation variants, and why the platform is just another set of Kubernetes objects.

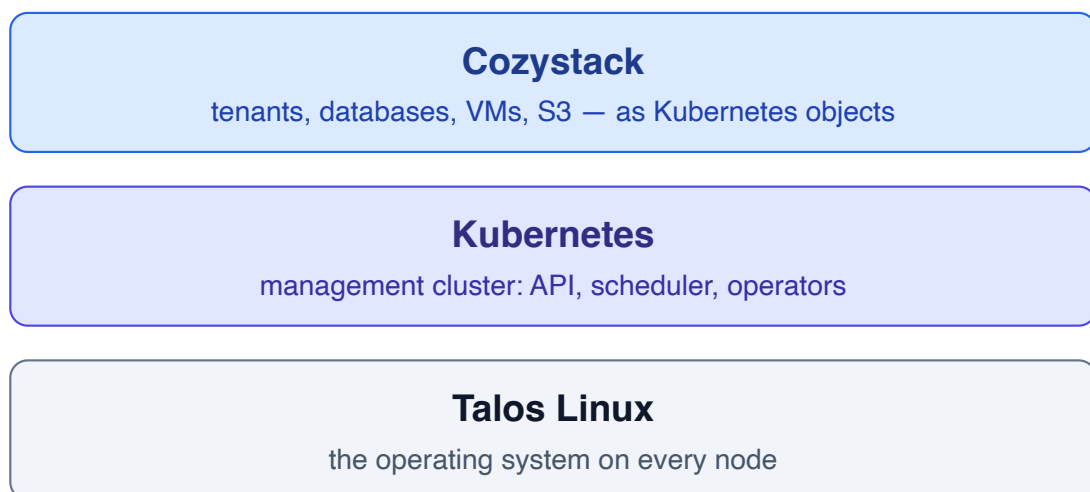
Let's start with the question the exam asks more often than any other: **what is Cozystack by nature?** Not "what is it for", but what exactly it is made of.

The short answer: it is Kubernetes with new object types added. Not a fork, not a layer on top of someone else's API, not a separate management server. When you ask the platform for a database, you create an object — just like a pod or a service, except it is called Postgres. From there an operator program watches it and does everything else.

It is worth getting this into your head before anything else, because almost everything else follows from it. Does `kubectl` work? It does. Does Terraform work? It does. Are permissions granted through ordinary RBAC? Yes, ordinary RBAC.

Three layers

The platform is built from three floors, and mixing up their order is the most common mistake on the exam.



Bottom to top: Talos → Kubernetes → Cozystack. Each layer stands on the one below.

Talos Linux is the node operating system (node OS). What sets it apart is that it has no command line: no SSH, no console, no package manager. It is configured declaratively, through its own API: you send a description of the desired state, and the node brings

itself into line with it. The root filesystem is read-only.

For an administrator used to logging into a server and tweaking something, this feels uncomfortable at first. The point is elsewhere: if you cannot log in and tweak things, the node's state does not drift over time. Two nodes built from the same description will still be identical a year later.

What gets installed is not stock Talos but **the platform's own image**: it adds the DRBD kernel modules — LINSTOR replication depends on them — and ZFS. The regular image does not have them.

Kubernetes is the management cluster, running on top of Talos. Ordinary, unmodified: the same API, the same controllers.

Two different concepts are easy to confuse here. There is one management cluster — that is the platform itself. The clusters that tenants order (tenant clusters) are already its product: their control plane runs as pods inside the management cluster.

Cozystack is a set of components installed into that cluster. They are what add the new object types and watch over them.

Installation in three stages

The exam asks not about commands but about the order and meaning of the stages.

1. **Talos on the nodes.** The machines boot from the platform image and receive a machine configuration. There are three ways to do it: boot-to-talos — an installer that writes Talos to disk from an already running Linux, booting from an **ISO**, and network boot via **PXE**. PXE is served by two temporary containers: **Matchbox** serves the image over HTTP, and **dnsmasq** provides DHCP and TFTP.
2. **Kubernetes.** The management cluster is assembled from these nodes. The recommended tool is **Talm**, the platform's own Talos configuration manager with Helm-style templates.
3. **Cozystack.** The platform itself is installed into the cluster, and from there it deploys its components on its own. It is installed with the `cozy-installer` chart into the `cozy-system` namespace, and then a single YAML is applied — the **Platform Package**. This is the only point of configuration: the platform domain, the apiserver address, address ranges, and the chosen installation variant.

The third stage works in an interesting way: the platform does not install components from a list; it describes the desired composition and hands it to **FluxCD**. From then on Flux brings the cluster into line with the description itself and keeps watching that the composition does not drift. Upgrading the platform means changing the version in the description, not rebuilding by hand.

Flux installs each platform component as a separate **release** — the object is called `HeLMReLease`, abbreviated to `hr` in commands. So the question “is the platform up” is a question about releases, not about pods:

```
kubectL get hr -A
```

All of them in the `READY True` state — the installation is complete.

Four installation variants

The platform is not installed as a whole but as one of four sets. You need to know the names **verbatim** — and here is why this is not nitpicking: in version 1.5 they were renamed, and the old names (`paas-full`, `distro-full` and the like) live on in other people’s articles. It is easy to mix them up.

Variant	What it is	When it is your case
<code>isp-full</code>	the whole platform on Talos	your own hardware, you need everything at once
<code>isp-full-generic</code>	the same, but on someone else’s Kubernetes	you already have a cluster: <code>k3s</code> , <code>kubeadm</code> , <code>RKE2</code>
<code>isp-hosted</code>	platform services without networking, storage and virtualization	on top of managed Kubernetes, where the provider supplies networking and disks
<code>default</code>	only package sources, nothing enabled	you assemble the platform yourself, component by component

It is easier to remember as a single phrase: `isp-full` — everything on Talos, `-generic` — the same on other distributions, `-hosted` — only the platform, the infrastructure underneath is someone else’s, `default` — from scratch and by hand.

You choose once, at installation. Changing the set on a running platform is not a switch; it is a reinstallation.

The variant sets the default composition, and fine-tuning is done with **packages**. Every platform component is a package named `cozystack.<component>`, and you can see their state with a single command:

```
kubectl get package
```

Two keys in the Platform Package let you adjust the set: `bundles.enabledPackages` adds packages on top of the variant, `bundles.disabledPackages` removes them. A caveat that the exam asks about: packages can be disabled only **before installation**. A component will not be removed from a running platform this way — you will have to remove it by hand through Helm.

What it consists of

There are many components, but the exam asks **who is responsible for what**, not for the full list. Keep this map in your head:

Task	Component
Virtual machines	KubeVirt
Control plane of tenant clusters	Kamaji
Block storage	LINSTOR / DRBD
Object storage (S3)	SeaweedFS
Pod networking	Cilium
Tenant networking, VPC	Kube-OVN
External addresses	MetaLB
Metrics	VictoriaMetrics
Logs	VictoriaLogs

Task	Component
Dashboards	Grafana
Configuration delivery	FluxCD
Sign-in with user accounts	Keycloak

The row about **Kamaji** is worth memorizing separately: it runs the control plane of tenant clusters as pods right in the management cluster — a tenant does not need separate machines for control-plane nodes.

Note what is **not** on the list: Prometheus and Loki. They are often slipped into answer options as a trap — metrics and logs here are collected by VictoriaMetrics and VictoriaLogs.

Hardware requirements

The numbers are asked verbatim, so you will have to learn the minimum profile as it is: **three nodes**, each with **8 cores**, **24 GB** of memory and **two disks** — 50 GB for the system and 256 GB for data. Two disks are not a whim here: the second one is given to LINSTOR. Fewer than three nodes gives fault tolerance neither to storage nor to the control plane.

The network is specified no less strictly: all nodes in **one L2 segment**, latency between them **under 10 ms** round-trip time (RTT).

If the nodes are themselves virtual — a lab in vSphere or Proxmox — enable nested virtualization and CPU flag passthrough. On bare metal this is not needed. A home lab built to this minimum is a legitimate way to go through the whole getting-started guide.

The platform is assembled. Next it has to be divided among someone — that is what the next lesson is about.

What the exam will ask

- The order of layers from bottom to top: Talos → Kubernetes → Cozystack.

- That Talos is managed through an API and has no SSH, and the root is read-only.
- That the platform extends Kubernetes with its own object types rather than replacing it.
- The names of the four installation variants (bundles) — verbatim.
- Who is responsible for what: KubeVirt, LINSTOR, SeaweedFS, Cilium, Kube-OVN, MetalLB.
- That metrics are stored by VictoriaMetrics, not Prometheus.
- That Kamaji runs the control plane of tenant clusters as pods in the management cluster.
- How the management cluster differs from tenant clusters.
- That packages are named `cozystack.<component>`, and `kubectL get package` lists them.
- The keys `bundles.enabledPackages` and `bundles.disabledPackages`; disabling works only before installation.
- That Talm is the recommended tool for the initial Talos configuration and cluster assembly.
- Why the platform has its own Talos image: the DRBD and ZFS kernel modules.
- Ways to install Talos: boot-to-talos, ISO, PXE (Matchbox serves the image, dnsmasq provides DHCP and TFTP).
- That the platform is installed by `cozy-installer` into `cozy-system` and configured by the Platform Package.
- Minimum hardware: 3 nodes, 8 cores, 24 GB, 50 and 256 GB disks, one L2 segment, latency under 10 ms.
- That installation readiness is checked by the state of the FluxCD releases.

Learn more: [platform overview](#) · [installation](#) · [getting started](#)

2. Tenants and access

How the platform divides itself among tenants, where quotas come from, and why a tenant is more than just a namespace.

The second-heaviest topic of the exam — and arguably the most useful in practice. If the first lesson explained what the platform is made of, this one explains how it is divided among people.

A tenant is an object, not a folder

In familiar Kubernetes, isolation begins and ends with a namespace. Here it is different: **a tenant is a platform object**, and creating one brings a whole set of consequences with it.

You create it the same way as everything else:

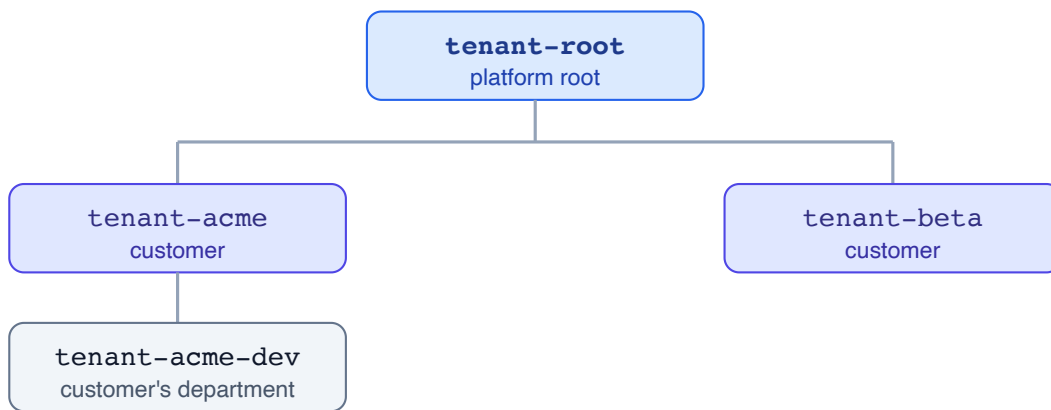
```
apiVersion: apps.cozystack.io/v1alpha1
kind: Tenant
metadata:
  name: acme
  namespace: tenant-root
```

In response, the platform creates a namespace, grants permissions, applies network policies, sets up quotas and, if requested, brings up its own monitoring and storage inside.

The tenant also gets its own domain, built by the rule <name>.<parent domain>. The platform lives at `cloud.example.com` — so tenant `alpha` gets `alpha.cloud.example.com`, and its child `beta` gets `beta.alpha.cloud.example.com`. If needed, the domain can be overridden manually.

Tenants nest inside each other

This is what sets the model apart from a flat list of namespaces: tenants form a tree.



A tenant lives inside its parent. The namespace name is made of the word tenant- and the name.

The naming rule is asked about, and it is a little trickier than it looks: the namespace name is built from the word tenant- and **the whole chain of ancestors joined by hyphens, except the root.**

Tenant path	Namespace
root/acme	tenant-acme
root/alpha/beta	tenant-alpha-beta
root/a/b/c	tenant-a-b-c

The root does not make it into the name — tenant-root-alpha-beta would be a wrong answer.

The same rule gives a restriction on names: **a hyphen in a tenant name is forbidden**, because it is reserved as the ancestor separator. There is no tenant acme-dev — there is dev inside acme, and it lives in tenant-acme-dev.

In the object itself these two names sit in different fields, and the difference between them is asked about too: `metadata.namespace` is the **parent's** namespace, where the CR itself lives, while the tenant's own namespace is written by the platform into `status.namespace`. That also gives you the way to list the children — `kubectl get tenants -n tenant-alpha` shows the CRs created inside alpha.

Why this matters: a provider hands a tenant to a customer, and the customer divides it among its own departments — without coming to the provider for every new environment.

What is inherited and what is enabled

A tenant has exactly four switches, and the exam asks for their literal names: `etcd` — its own `etcd` cluster, `monitoring` — its own monitoring, `ingress` — its own ingress controller with TLS, `seaweedfs` — its own S3 object storage. Each can be enabled or left disabled.

The key idea the exam likes to test: **if a service is disabled, the tenant goes up the tree to the nearest ancestor that has it enabled**. Not necessarily to the parent and not necessarily to the root — to the nearest one. It is not left without the service; it inherits it. A tenant without its own monitoring sends metrics to the parent's monitoring; a tenant without its own storage puts files into the parent's.

Enabling your own makes sense when you need real isolation — for example, a customer must not see other customers' metrics even theoretically. The cost is resources: every enabled service is processes that actually run.

Quotas and overcommit

A tenant is assigned a limit on CPU and memory:

```
spec:
  resourceQuotas:
    cpu: 16
    memory: 32Gi
```

Resource sizes are set with a **preset** in the format `<series>.<size>`. The series fixes the CPU-to-memory ratio: `t1` — 1:0.5, `c1` — 1:1, `s1` — 1:2, `u1` — 1:4, `m1` — 1:8. Sizes range from nano to 4xlarge, so `u1.medium` is the universal series in a medium size.

If a resources block is written explicitly next to the preset, it wins:

```
resources:
  cpu: 4
  memory: 8Gi
```

And the trap that all of this is asked about for: tenant and application presets are **not the same thing** as KubeVirt instance types for virtual machines, with the series U, Q, CX, M and RT. The string `u1.medium` is valid in both systems and means different things, so check which object the question is about.

And now the thing practically everyone trips over — **overcommit** (`cpuAllocationRatio`, **10** by default).

It works like this: when you run a workload with a limit of four CPUs, the platform asks the scheduler not for four, but for **the limit divided by the ratio** — that is, four tenths. A small amount is guaranteed, and a lot can be taken when needed.

The formula is short, and it is worth memorizing verbatim:

$$\text{request} = \text{limit} \div \text{cpuAllocationRatio}$$

The point is that applications almost never consume their limit. Handing out guarantees at the upper bound means keeping half of the hardware idle. But if you do not know about the ratio, the scheduler's behavior seems senseless: you asked for four cores, and less than half of one is reserved.

It matters where this ratio lives: it is configured at the level of **the whole platform**, not in an individual tenant. One for everyone — do not pick it as a tenant property in the answer options.

Isolation

By default tenants cannot see each other: network policies forbid traffic between namespaces. And this is not a switch — **isolation cannot be disabled**. The `isolated` flag existed before version 1.0 and has been removed; if it shows up in the answer options, it is a trap.

Pods are cut off not only from their neighbors. By default they cannot reach either kube-apiserver or the tenant's own etcd. If access is needed, it is opened selectively, with labels on the pod:

```
policy.cozystack.io/allow-to-apiserver: "true"
policy.cozystack.io/allow-to-etcd: "true"
```

The value is the string `"true"`, and that is asked about too.

How people get inside

Three paths, and this is asked about too.

Dashboard — the platform's web interface. Sign-in is through **Keycloak**, that is, with a corporate account rather than a separate password.

kubeconfig — the access file issued to a tenant. Inside it is not a certificate but a call to an external program: on the first request `kubectl` opens a browser, you sign in through Keycloak, and the issued token is valid until it expires. This, by the way, is why you need to install `kubelogin`: without it `kubectl` does not know how to log in.

Terraform — the same API, only declaratively.

Where the kubeconfig file itself comes from

A tenant is not handed a ready-made kubeconfig file — the platform administrator builds it with a script from the documentation. You do not need to know that script by heart, but **what it builds the file from** is asked about.

Every tenant has a secret with the same name in its namespace: for tenant `alpha` it is the secret `alpha` in the `tenant-alpha` namespace. The script takes three things from it:

What is taken	From which field	Why
access token	token	the tenant presents itself to the API server with it
certificate authority certificate	ca.crt	<code>kubectl</code> uses it to verify the server is genuine
default namespace	namespace	so you do not have to write <code>-n</code> in every command

The address of the API server itself the script takes not from the secret but from the current kubeconfig of the administrator who runs the script. In total, the file is built from **the token, the CA certificate and the server address** — and in this variant neither Keycloak nor `kubelogin` is required: the token is already inside the file.

An important consequence follows: **a tenant kubeconfig is a secret in exactly the same sense as a password**. Whoever gets the file gets the tenant's permissions too, until the token is revoked.

Permissions inside a tenant are granted with ordinary Kubernetes roles. The platform has no separate permission system, and that is the answer to a popular trick question.

The tenant is created, quotas are assigned, people are let inside. Now they need to order something.

What the exam will ask

- That a tenant is a platform object, not just a namespace.
- That the namespace name = tenant- plus the chain of ancestors joined by hyphens, without the root.
- That a hyphen in a tenant name is forbidden.
- That tenants nest inside each other, and a nested one is created inside its parent.
- That a disabled service is inherited from the nearest ancestor that has it enabled — not necessarily from the parent.
- The names of the four switches verbatim: etcd, monitoring, ingress, seaweedfs.
- That a tenant's domain is built as <name>.<parent domain>.
- That `metadata.namespace` is the parent's namespace, and `status.namespace` is the tenant's own.
- That the script from the documentation builds a tenant kubeconfig from **the token, the CA certificate and the server address**: the first two from the tenant's secret of the same name, the address from the administrator's kubeconfig.
- The preset format <series>.<size>: t1 (1:0.5), c1 (1:1), s1 (1:2), u1 (1:4), m1 (1:8); sizes from nano to 4xlarge.
- That an explicitly set resources block overrides the preset.
- That tenant presets are not KubeVirt instance types (U, O, CX, M, RT): `u1.medium` exists in both systems.
- That a workload's request = its limit divided by `cpuAllocationRatio` (10 by default).

- That `cpuAllocationRatio` is configured at the platform level, not for an individual tenant.
- That isolation cannot be disabled, and the `isolated` flag was removed in version 1.0.
- The label names verbatim: `policy.cozystack.io/allow-to-apiserver` and `allow-to-etcd`; the value is the string `"true"`.
- That sign-in goes through Keycloak, and permissions through ordinary RBAC.

Learn more: [tenants](#) · [platform operations](#)

3. Managed applications catalog

What you can order from the platform, what happens after you click the button, and why it is always the same mechanism.

The catalog is usually the reason people take the platform in the first place. Instead of “please set up PostgreSQL for us”, a person opens a list and orders a database the way they would order a virtual machine.

What is on the list

The catalog is split into four groups, and the exam names them in English: Databases, Messaging, Platform services, and Networking services.

Group	Items
Databases	PostgreSQL, MariaDB, MongoDB, ClickHouse, FoundationDB, Redis, Qdrant
Messaging	Kafka, NATS, RabbitMQ
Platform services	Harbor, OpenBao, Bucket, Tenant, Monitoring, Etcd, Ingress
Networking services	VPN, TCP balancer, HTTP cache, virtual-router, VPC

Virtual machines and Kubernetes clusters live in the same API group, but the exam assigns them to other domains — virtualization and multi-tenancy — rather than to the catalog.

One item in platform services comes as a surprise: **Tenant is a catalog application too**. That is exactly how nested tenants are created — an administrator orders a child tenant from the catalog, just as they would order a database.

The list grows from version to version, and there is no point memorizing all of it. What gets asked is something else — **that they are all built the same way**.

One mechanism for everything

This is worth understanding once, so you never have to come back to it.



The same path for a database, a queue, a virtual machine and a cluster.

You create an object. The platform creates a `HelmRelease` for it, Flux deploys the chart — and the chart in turn brings the **operator**, which from then on runs your database: it watches replication, takes backups, and switches the primary copy over on failure.

The platform does not write its own database engines — it uses well-known operators. Their names come up in questions:

Service	Operator
PostgreSQL	CloudNativePG
Kafka	Strimzi
MongoDB	Percona
ClickHouse	Altinity

Remember the order of the links exactly this way: the operator comes **from** the chart, it does not precede it.

Two practical conclusions follow from this, and both show up in questions. First: **the dashboard does nothing special** — clicking the button assembles exactly the same object and sends it to the API. Second: if a service has not come up, look at the `HelmRelease` instead of guessing from the pods.

Which application types exist on a particular cluster is shown by a single command:

```
kubectl api-resources | grep apps.cozystack
```

The platform writes the result back into the object itself — into `status.conditions`. A healthy application has `Ready: True` there; if it does not, go further down the chain, to the `helmRelease`.

Three paths to the same thing

The dashboard is a form with fields. The fields come from the application's definition, so the list of parameters always matches the platform version.

kubectl is the same object as text. This path matters not because it is more convenient, but because the definition can be reviewed, stored in Git and rolled back. A button click cannot be rolled back.

Terraform is for those whose infrastructure is already described in it. The official provider is called `cozystack/cozystack`; the name is worth remembering, it gets asked.

And a warning common to all three paths: `kubectl delete` on an application object wipes out the application entirely — together with its pods and data. Treat this command the same way you would treat deleting a database.

What you set when ordering

Each service has its own set of fields, but three things are almost always there.

Size (`resourcesPreset`) — how many resources to give. It is set not with numbers but with a ready-made preset such as `t1.micro` or `u1.medium`: the platform substitutes concrete values behind it.

Storage (`storageClass`) — how much space and of which class. `replicated` keeps several copies on different nodes; `local` is faster but lives on a single one.

Users and databases — the platform creates them itself. That is exactly why the schema file you apply afterwards contains no commands to create the database and the user: they have already been run.

The platform generates passwords itself and puts them in a secret. In the dashboard it is visible on the Secrets tab of the application itself.

High availability is not free

When ordering a service, you choose the number of replicas. One replica is a training setup: the node goes away, and the service goes with it. With two or more, the platform itself keeps track of which one is primary and switches over on failure.

This is also where a common trap lives: **service high availability and data safety are different things**. Replicas save you from a node failure, but not from a dropped table. For the latter you need backups, and they have a lesson of their own.

What version 1.5 brought

Three facts from this release are asked about directly. Encryption of client connections (TLS) arrived for four services: Kafka, NATS, Qdrant and PostgreSQL. Backups of managed applications started working out of the box — previously an administrator first had to set up storage for them. And ApplicationDefinition appeared: the mechanism registers a new application type in the catalog on top of a Helm chart, and that type immediately gets its own API object, a form in the dashboard and the same chain with a HelmRelease. This is how organizations add their own services to the shared catalog — the catalog is not a closed list.

We have covered the catalog, but one of its items — virtual machines — deserves a separate look: it has a model of its own, and habits from vSphere let you down here.

What the exam will ask

- The four catalog groups: Databases, Messaging, Platform services, Networking services — and what belongs where.
- That Tenant is itself a catalog item, and nested tenants are created through it.
- The operators by name: CloudNativePG — PostgreSQL, Strimzi — Kafka, Percona — MongoDB, Altinity — ClickHouse.
- That ordering any service means an object in the API, and the dashboard merely assembles it for you.

- That the list of types is shown by `kubectl api-resources | grep apps.cozystack`.
- That readiness is read from `status.conditions` — it should show `Ready: True`.
- That `kubectl delete` on the object deletes the application along with its data.
- The Terraform provider name is `cozystack/cozystack`.
- What is new in 1.5: TLS for Kafka, NATS, Qdrant and PostgreSQL; backups out of the box; `ApplicationDefinition` for extending the catalog.
- The chain: object → `HelmRelease` → chart → operator → running pods.
- That troubleshooting starts with the state of the `HelmRelease`.
- That the platform creates users and databases and puts passwords in a secret.
- The difference between `replicated` and `local`.
- That the number of replicas protects against a node failure but does not replace backups.

Further reading: [managed applications](#) · [Kubernetes clusters](#)

4. Virtual machines

How a virtual machine becomes an ordinary Kubernetes workload, why the disk is separate from the machine, and where the honest limits are.

This is the topic where a VMware administrator has the easiest time — and also where they most often get caught, because familiar analogies only half work here.

The machine as an ordinary workload

Virtualization is provided by **KubeVirt**. Its idea is this: a virtual machine is a process that runs inside a pod and is managed like everything else in the cluster.

From this follows something that sounds unusual to a person from the vSphere world. The Kubernetes scheduler picks a node for a machine the same way it does for an application. Resources are accounted with the same requests and limits. Permissions are granted with the same RBAC.

The hypervisor, meanwhile, is a real one — KVM, the same as in any Linux.

Two objects instead of one

This is the main difference from the familiar model, and it gets asked.



The disk outlives the machine: delete the machine and the disk stays.
It can be attached to another one or deleted separately.

In vSphere a disk is a property of the machine. Here it is an object in its own right.

The disk is created separately and lives a life of its own. It gets its contents in one of four ways, and they are worth telling apart: download from a URL (`http`), a clone of a prepared platform image (`image`), an upload of a file from your own machine (`upload`),

and the empty source `{}` — a blank disk with no contents, for when you just need space for data.

The third object of the model is **VMImage**, a prepared image that the platform stores itself. A team publishes, say, Ubuntu once, and new disks clone it via `source.image.name` instead of pulling the file from the internet every time.

Underneath, all of this is ordinary Kubernetes storage: the disk exists as a `PersistentVolumeClaim`, and the image is poured into it by the **CDI** component (Containerized Data Importer) through a `DataVolume` object. Nobody touches these by hand here, but those are exactly what you will meet on the lower layers.

The machine's size is set not with numbers but with a named **instance type** such as `u1.medium` — this is KubeVirt's own mechanism, with series U, O, CX, M and RT. Do not confuse it with tenant and managed application presets: the string `u1.medium` is valid in both places, but the systems are different. Look at what the question is sizing: a tenant and an application take a preset, a `VMInstance` takes an instance type.

How to get inside

This is where habits end and the specifics begin.

cloud-init is the standard mechanism of cloud images: on first boot the system reads a description and configures itself. Password, keys, packages, commands. It is the closest analog of a Customization Specification, only described as text next to the machine itself.

virtctl is a separate utility for the console, the screen and port forwarding. There are three ways in, and they refer to the machine by its bare name:

```
virtctl console <name>           # text console over the serial port
virtctl vnc <name>                # graphical screen, also the way into Windows
virtctl ssh <user>@<name>        # everyday work with a Linux guest
```

The scenario from the questions goes like this: networking inside the machine is broken, SSH does not respond — what will work? **virtctl console**: the serial console does not depend on the guest network, which is why it remains the last resort, including when investigating early boot.

A separate caveat about restricted access. A tenant is granted the console as a subresource of the **running instance**, not of the machine object as a whole, and under such permissions the target is named with a type prefix: `virtctl console --namespace=tenant-acme vmi/vm-instance-app`. A bare name in this case points to the wrong thing and returns a denial.

What it can and cannot do

The exam likes questions about capabilities — and here it is important not to confuse what the platform does not do with what it does differently.

Live migration exists. A running machine moves to another node without stopping — this is used when a node has to be taken down for maintenance.

Snapshots exist. They are provided by the storage layer.

GPU passthrough exists, and it is configured automatically. The list of devices that are allowed to be passed through is set by the platform, not by the tenant.

It is also worth remembering **what exactly changed in version 1.5** — a separate question is built on it. Passthrough itself existed before, but node preparation had to be done by hand: labeling the nodes with GPUs and writing the allowed devices directly into the KubeVirt settings with `kubectl patch`. In 1.5, exactly this was automated. A node group in which GPUs are declared now gets the required label by itself, and the platform fills in the list of allowed devices by itself. The statement “passthrough only appeared in 1.5” is wrong — what appeared is **automatic node preparation**, not the capability itself.

What truly does not exist is **automatic load balancing between nodes**. In vSphere this is DRS’s job: it decides on its own that a machine should be moved, and moves it. There is nothing like that here. Capacity planning and moving machines during maintenance are done deliberately, not on their own.

The difference is subtle, but it is exactly what questions are built on: migration as a **tool** exists, migration as **always-on automation** does not.

If a node dies

The question every team asks before moving off vSphere: will the machines restart on another node by themselves? The honest answer for version 1.5 is **no**, and it is also the correct answer on the exam.

The reason is not something left unfinished, but the design. When a node stops responding, the cluster does not know what happened to it: whether it shut down, or lost its network and keeps running. For an ordinary application there is no difference — an extra copy bothers no one. For a virtual machine with a disk the difference is fundamental: starting a second copy of the same machine while the first may still be writing to the disk means data corruption. That is why safe automatic restart requires **fencing** — guaranteed shutdown of the suspect node before the machine is brought up again. The distribution ships no such mechanism, and recovery after a node failure still involves manual steps.

Live migration does not help here and cannot help: it moves a **running** machine off a **live** node. It is a planned evacuation before maintenance, not a response to an outage.

Separately — so as not to mix two different things. In **tenant Kubernetes clusters** the worker nodes are virtual machines too, but their node health checks and replacement of broken ones do work and are enabled by default. Only this is not “restarting the same machine” but **replacement**: the faulty machine is deleted and a new, clean one is created in its place. For a tenant cluster node this is fine — its state is not stored in the cluster. For your own virtual machine with data on its disk it is unacceptable, and that is exactly why the same mechanism is not applied to it.

The machine is running. Packets reach it over the network — and there are four components there that are easy to mix up with one another.

What the exam will ask

- That virtualization is provided by KubeVirt, and a machine is an ordinary cluster workload.
- That the disk and the machine are two separate objects, and the disk outlives the machine.
- The four disk content sources: `http`, `image`, `upload` and the empty `{}` — a blank disk.
- That `VMImage` is the third object of the model: a prepared image from which disks are cloned.
- That the disk exists as a `PVC`, and `CDI` pours the image into it through a `DataVolume`.
- That the machine's size is set by a KubeVirt instance type (series `U/O/CX/M/RT`), not by a tenant preset.

- That initial configuration goes through cloud-init.
- Three ways in: `virtctl console`, `virtctl vnc`, `virtctl ssh` — and that with broken networking in the machine, the console will work.
- That the `vmi/` prefix is needed under restricted tenant permissions — because access is to the subresource of the running instance.
- That live migration, snapshots and GPU passthrough exist.
- That there is no automatic balancing between nodes like DRS.
- That 1.5 automated **node preparation** for GPU passthrough — node labeling and the list of allowed devices — not passthrough itself.
- That there is **no** automatic restart of a virtual machine on another node after a failure: that requires fencing, which the distribution does not ship, and recovery still involves manual steps.

Further reading: [virtualization](#) · [storage](#)

5. Networking

Four components, each with its own job: who carries packets inside, who separates tenant networks, who hands out external addresses and who lets HTTP in.

The networking part looks complicated until you break it down by job. There are four components, and each has its own work to do — the exam asks about the division of roles, not about settings.

Ingress / Gateway

lets HTTP in from outside, separately for each tenant

MetalLB

hands out external addresses on your own hardware, without a cloud load balancer

Kube-OVN

separate tenant networks, VPC, address assignment

Cilium

carries packets between pods, enforces network policies

Bottom to top: from packets between pods to the entry point from outside.

Cilium — the foundation

It makes sure pods can see each other, and it handles network policies. It runs on **eBPF** — a technology that lets programs run directly in the Linux kernel, without switching to user space for every packet.

The practical consequence: network rules are applied in the kernel, without long iptables chains, and they do not degrade as their number grows.

It is Cilium that enforces the isolation between tenants discussed in the second lesson.

There is also a third role that is easy to miss: Cilium **replaces kube-proxy** (kube-proxy replacement). Normally Kubernetes services are handled by kube-proxy via iptables; here the right destination is looked up in a hash table, and the cost is the same no matter how many services there are. On a platform with hundreds of tenants this is noticeable.

The division of labor between it and Kube-OVN is worth remembering, because both are called “the network”: **Cilium is the primary CNI**; it gives pods their network and enforces policies for everyone. **Kube-OVN runs on top** and is responsible for the overlay network, address assignment and tenant VPCs.

Kube-OVN — tenant networks

Kube-OVN builds an **overlay network** (overl`a`y) on top of the nodes’ physical network and adds two properties to it that the exam asks about more often than anything else.

Centralized address assignment (centralized IPAM): addresses are handed out from one place rather than separately on each node, and by default the whole cluster lives in one shared pod range.

Stable pod addresses (stable pod IPs): a pod keeps its address when it moves to another node. For a container this is a convenience; for a virtual machine it is a necessity: guest systems, licences and firewall rules are usually tied to the address, and changing the address on every migration would break them.

Kube-OVN also provides **VPCs** — private networks with their own address space. Both VPC and the virtual router (virtual-router) are items in the managed applications catalog: a tenant orders them the same way as a database.

The closest analogy from the familiar world is the VPC at cloud providers, or NSX networks.

MetalLB — external addresses

In a public cloud, a service of type LoadBalancer gets its address from the provider. On your own hardware there is no provider, and without MetalLB such a service hangs in pending forever.

MetalLB holds a pool of addresses, hands them out to services and then announces them to the network — either via ARP, or via BGP if the network is built on routing. Starting with version 1.5, the BGP side is handled by **FRR-K8s** (FRR-K8s) — it is the one that announces routes to the physical routers.

It is what you need for a virtual machine or an application to get an address reachable from outside the cluster.

Ingress — the entry point for HTTP

For web applications, giving each one its own address is wasteful. **Ingress** accepts requests on a shared address and distributes them by host names and paths.

A platform specific: **each tenant has its own entry point (ingress)**. This is not a shared controller for the whole cluster — a tenant with ingress enabled gets its own ingress-nginx, with its own rules, its own external address from MetalLB and its own certificates. A tenant without one uses its parent's, just as with the other services.

TenantGateway — the Gateway API path

Kubernetes is gradually moving from the old Ingress to the **Gateway API** — a more expressive standard for routing traffic. In the platform it appeared in version 1.5 as the TenantGateway object.

Packets through it are carried by **Cilium**: there is no nginx on this path at all. And it is not a replacement for ingress-nginx but an alternative — in 1.5 both options coexist.

TenantGateway can issue certificates in two validation modes. **HTTP-01** — the certificate authority fetches a token over plain HTTP, so the domain must be visible from the internet. **DNS-01** — validation via a DNS record; only this mode works for wildcard certificates like *.apps.example.com and for entry points that are not exposed to the outside.

Names and certificates

Three helpers that usually go unnoticed as long as they work.

CoreDNS resolves names inside the cluster — that is why application settings say postgres-db-rw rather than an address: the name survives the database moving to another node, the address does not.

ExternalDNS watches published services and entry points and creates records for them at your DNS provider by itself. An application gets published, the name starts resolving — no need to file a ticket with the network team.

cert-manager issues certificates and renews them by itself, without reminders.

All of this works right up until the first failure. How to find out about a failure and what is worth preparing in advance is the next lesson.

What the exam will ask

- That Cilium carries packets between pods and runs on eBPF.
- That Cilium replaces kube-proxy.
- That Kube-OVN provides separate tenant networks and VPCs.
- That Kube-OVN has centralized address assignment and one shared pod range per cluster.
- That a pod keeps its address when moving to another node — and why this is critical for VMs.
- That VPC and the virtual router (virtual-router) are items in the managed applications catalog.
- That MetalLB hands out external addresses on your own hardware, announcing them via ARP or BGP.
- That since version 1.5, BGP in MetalLB is handled by FRR-K8s.
- That the HTTP entry point — ingress-nginx — is set up separately for each tenant.
- That TenantGateway appeared in 1.5, it is the Gateway API, and its packets are carried by Cilium.
- Two certificate issuance modes: HTTP-01 and DNS-01; wildcard — DNS-01 only.
- That DNS records for published applications are created by ExternalDNS.
- That names inside the cluster are resolved by CoreDNS, and certificates are issued by cert-manager.
- Why application settings use service names rather than addresses.

More: [platform networking](#)

6. Observability and backups

Who collects metrics and logs, where to look at them, and how a backup differs from fault tolerance.

Two topics in one lesson, because both are about the same thing: what to do when something has gone wrong — and what you managed to prepare in advance.

Metrics and logs

The stack here is not the one people expect out of habit, and this is the exam's favorite trap.

What is collected	With what
Metrics	VictoriaMetrics
Logs	VictoriaLogs
Dashboards	Grafana
Alerts	Alerta

There is no Prometheus, Loki or Elasticsearch in the platform. VictoriaMetrics does understand the PromQL query language, though — so familiar queries work, while the storage is different: more compact and cheaper at large volumes.

Metrics are collected by the **vmagent** agent, which lives next to the workload and sends what it collects to the storage. The storage itself is deployed as a **VMCluster** — a clustered VictoriaMetrics installation managed by an operator. Logs are collected by **fluent-bit** — the same role, only for text.

The alerting chain is short, and the exam asks about its order: **VMAAlert** evaluates rules against the metrics in VictoriaMetrics → **Alerta** gathers the firings in one place, removes duplicates and routes them → email, SMS, messengers. Alerta is there so that a single incident does not arrive as eight emails from eight sources.

Grafana ships with ready-made dashboards, and the exam asks about them by group name: **Cluster Overview** — the health of the cluster as a whole, **Node Metrics** — CPU, memory, disk and network per node, **ETCD** — the state of the etcd store, **Storage** — the health of LINSTOR and SeaweedFS, **Tenant Applications** — workloads and managed applications per tenant. There are also dashboards for entry-point traffic and for managed databases.



Logs follow the same path, only with fluent-bit instead of vmagent and VictoriaLogs instead of VictoriaMetrics.

Remember inheritance from the second lesson: a tenant without its own monitoring sends metrics to its parent. But turning collection on **after the fact is useless** — there is nowhere to get records for the past from, so this should be decided when the cluster is created, not when a graph is suddenly needed.

Backups

There are two different mechanisms here, and mixing them up is a sure way to get a question wrong.

Backups of managed services (backup). **Four objects** are at work here, and mixing them up is a sure way to get it wrong:

Object	What it does
BackupClass	where and how to store backups. Created by the platform administrator, applies to the whole cluster
Plan	the schedule: take backups on such-and-such a timetable
BackupJob	a one-off run, here and now

Object	What it does
Backup	the result — the backup itself

The key pair is Plan and BackupJob. You need a backup **every night** — that is a Plan. You need a backup **right now, before an upgrade** — that is a BackupJob. Restoring is a separate object, RestoreJob.

Starting with version 1.5, the platform has a ready-made cozy-default — a backup class that works right away, without configuring storage.

Velero. One level up: it backs up the cluster's own objects and volumes. This is about restoring the platform, not an individual database. Since version 1.5 it is not an option but a standard component — installed by default.

It is configured with two objects: BackupStorageLocation — where to store backups, and VolumeSnapshotLocation — where to keep volume snapshots. For a Kubernetes cluster inside a tenant, Velero is enabled with a single field: `spec.addons.velero.enabled`.

Where the boundary lies

The most valuable thing in this topic is understanding what backups do **not** do.

A backup of a managed application takes **only the data**. It does not include the application's HelmRelease, the CR object you created in the managed applications catalog, or the secrets created by the database operator.

Hence the restore rule: the target application must exist **before** you restore. A restore pours data into an existing database; it does not rebuild the application from scratch.

There are no incremental backups of virtual machines: the platform does not do changed block tracking, and every backup is a full one. If you are used to incremental chains, you will have to recalculate backup windows and storage capacity.

Backups are not fault tolerance. Replicas save you from a node failure, backups from deleted data. These are different troubles, and one does not replace the other.

A backup that has never been restored is not a backup but a hope. Restores have to be tried before they are needed.

And most importantly: a backup is kept in object storage. If that storage lives in the same cluster as the data, it will not save you from losing the whole cluster. For real protection, the storage must be outside.

We have covered what the platform is made of and what it can do. One last question remains — why it is built exactly this way.

What the exam will ask

- That metrics are stored by VictoriaMetrics and logs by VictoriaLogs. Not Prometheus and not Loki.
- That VictoriaMetrics is PromQL-compatible.
- That metrics are collected by vmagent and stored by VMCluster.
- The alerting order: VMAalert → Alerta → email, SMS, messengers.
- The standard dashboard groups: Cluster Overview, Node Metrics, ETCD, Storage, Tenant Applications.
- The four objects: BackupClass, Plan (schedule), BackupJob (one-off run), Backup (result).
- That a scheduled backup is a Plan, not a BackupJob.
- That cozy-default works out of the box since version 1.5.
- That Velero works at the platform level, not at the level of an individual service.
- That Velero became a standard component in version 1.5, and in a tenant cluster it is enabled with the `spec.addons.velero.enabled` field.
- That BackupStorageLocation sets where backups are stored, and VolumeSnapshotLocation where volume snapshots are kept.
- What a "data only" backup does not take: the HelmRelease, the CR object, the operator's secrets.
- That the target application must exist before the restore.
- That there are no incremental backups of virtual machines — every backup is a full one.
- That replicas and backups solve different problems.

More: [monitoring](#) · [cluster services](#)

7. How it all actually works

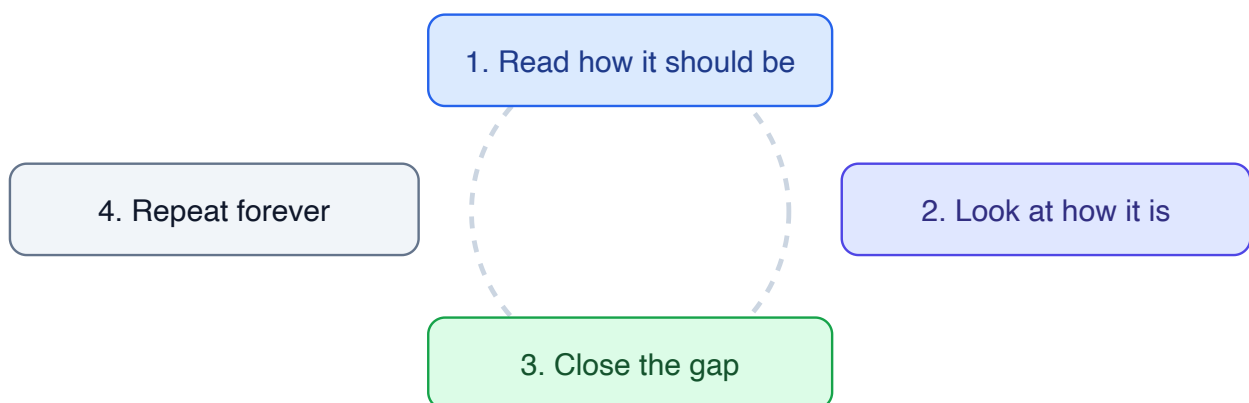
Desired state, operators and GitOps — the four ideas everything else rests on.

The last lesson is about ideas, not components. They underlie everything covered so far, and the questions about them are phrased simply: “why does it work this way”.

You describe the outcome, not the actions

The familiar approach to administration is a sequence of steps: install, configure, start, check. Here it is different: you describe **how things should be** — the desired state — and the system decides on its own what to do to get there.

The difference shows when something breaks. A script that ran yesterday will not help today: it has already been executed. A description of the desired state, on the other hand, is in effect all the time — if reality drifts away from it, reality is brought back.



This loop runs continuously — both in Kubernetes controllers and in the platform's operators.

Hence self-healing: a deleted replica of an application comes back not because someone noticed it was missing, but because reality diverged from the description.

What a manifest is made of

The description of the desired state lives in a YAML file called a **manifest**. Whatever object you describe — a tenant, a database, a virtual machine — there are always four top-level fields, and mixing them up in the exam is a bad idea.

Field	What it holds
apiVersion	which API version the type belongs to, for example apps.cozystack.io/v1alpha1
kind	the type itself: Tenant, Bucket, VMInstance
metadata	identity data: name, namespace, labels, annotations
spec	the desired state — everything the object should become

The fifth field, status, you never write: the controller fills it in, and it says how things actually are. Hence a simple rule for reading any object: **spec is your requirement, status is the platform's answer to it**. The reconciliation loop above is exactly the continuous bringing of one in line with the other.

```
apiVersion: apps.cozystack.io/v1alpha1
kind: Bucket
metadata:
  name: images           # what the object is called
  namespace: tenant-lab # where it lives
spec:                   # what it should become
  replicas: 2
```

New object types and operators

`kubectl get tenants` works even though Kubernetes itself has no tenants. It works because Cozystack **extends the Kubernetes API**, and the API server knows the Tenant type.

The API is extended in two different ways, and the exam distinguishes between them.

Defining your own type — a CRD (custom resource definition). You register a new type, and from then on its objects are stored and served by the regular API server along with the built-in ones. This is how, for example, backups (backups.cozystack.io) and gateways (gateway.cozystack.io) are implemented in the platform.

An aggregated API server. A separate program takes over an entire API group, and the main API server forwards requests to it. In Cozystack this is cozystack-api in the cozy-system namespace, and it is the one responsible for the most visible groups —

apps.cozystack.io (tenants, buckets, databases, clusters), core.cozystack.io, sdn.cozystack.io. You will not find a CRD named tenants.apps.cozystack.io in the cluster: this type is not registered, it is **served** by the aggregated server.

Which component is responsible for which group is visible with a single command — the SERVICE column shows either the program's address or the word Local, meaning "the regular API server, type from a CRD":

```
kubectl get apiservices | grep cozystack
```

But a type on its own does nothing: it is a record the cluster agrees to store. The work is done by an **operator** — a program that watches objects of its type and brings the world into line.

Hence the practical conclusion: if an object has been created and nothing happened, the question is not for the object but for the operator. Either it is not running, or it failed.

GitOps

Since state is described as text, the text can be stored in version control. Git then becomes the single source of truth, and a dedicated program makes sure the cluster matches it.

The platform uses **FluxCD** and applies this approach **to itself**: its components are described and deployed the same way you would deploy your own application.

One caveat, so as not to overstate things. The desired state of the platform itself is defined not by your repository but by the **Platform Package** — the YAML configuration of the installation; FluxCD pulls the charts from an OCI registry. An agent inside the cluster continuously brings the cluster to the described state, so a change made bypassing the description does not live long.

FluxCD has two objects you need to know by name. HelmRepository is the source the charts come from. HelmRelease (short name hr) is the statement "this chart, of this version, with these values, must be installed and stay installed".

How to temporarily disable reconciliation

Continuous reconciliation gets in the way in exactly one case: when you need to fix something by hand. A change made bypassing the description will be rolled back by the controller within a few seconds — that is what it exists for.

For this case HelmRelease has a switch, `spec.suspend`. For a release switched to suspend, the controller **stops reconciling**: it does not reinstall the chart, does not roll back changes, and does not touch the object at all. Meanwhile **the workload keeps running** — pods are not deleted, the application responds — and your manual changes are kept until reconciliation is switched back on.

```
kubectl patch hr <name> -n <namespace> --type merge -p '{"spec":{"suspend":true}}'
```

The reverse operation is `"suspend": false`. At the moment it is turned back on, the controller reconciles the object anew, and everything you fixed by hand bypassing the description is rolled back. That is why suspend is a diagnostic tool for the duration of an investigation, not a way to live with manual changes.

Helm

One object is one object. An application is usually a dozen: a deployment, a service, configuration, secrets, access rules.

Helm packages such a set into a **chart** — templates plus values. By changing the values, you get different installations from one package.

In the platform, Helm is not a recommendation but a load-bearing structure: every item in the managed applications catalog is a chart, and ordering a service deploys exactly that chart. The platform itself is also installed with a Helm chart — with a single `helm upgrade --install` command using the `cozy-installer` chart into the `cozy-system` namespace.

The result is the following chain, and it is worth memorizing in full:

object → HelmRelease → chart → operator → running pods

Everything the platform does goes through it. You ordered a database — it went through it. You created a tenant — it went through it. The platform installed itself — that too.

Kubernetes vocabulary that will be asked here too

This exam domain is described as “the seventh topic plus basic terminology”, so it is worth going over seven terms out loud. The English names are exactly the ones that will appear in the questions.

Object	What it is and why
Namespace	a named space that groups resources; each tenant has its own
Pod	the smallest unit of workload — one container or several together
Deployment	describes what is desired: which image, how many replicas, how to update
ReplicaSet	its executor: keeps the required number of pods; it is rarely touched by hand
Service	a stable name and address in front of a changing set of pods
PVC	a claim for persistent storage that survives a pod restart
Secret	stores sensitive data: passwords, tokens, keys

Service has three types: **ClusterIP** — the address is visible only inside the cluster, this is the default type; **NodePort** — opens a port on every node, good for testing; **LoadBalancer** — asks the platform for an external address; on your own hardware it is provided by MetalLB from lesson five.

And five commands the exam asks about by their exact spelling:

- `kubectl get pods -A` — all pods in all namespaces at once
- `kubectl describe pod <name>` — pod details and, most importantly, its events
- `kubectl api-resources` — which resource types the cluster knows at all, including those added by the platform
- `kubectl apply -f manifest.yaml --dry-run=server` — submit the manifest to the server for validation without saving anything
- `helm upgrade --install` — idempotent: installs if the release does not exist, upgrades if it does

What the exam will ask

- That it is the desired state that is described, not a sequence of actions.
- That a controller continuously compares the desired state with the actual one and closes the gap.

- That an object's desired state lives in `spec`, the actual state in `status`, while `apiVersion`, `kind` and `metadata` are responsible for the API version, the type and the name.
- That a CRD adds a type, while the work is done by an operator.
- That `kubectl get tenants` works because Cozystack extends the Kubernetes API: the `Tenant` type is served by the aggregated server `cozystack-api`, not by a CRD.
- That `suspend` on a `HelmRelease` stops reconciliation but does not delete the workload: pods keep running, manual changes are kept until reconciliation is turned back on.
- That the platform applies GitOps to itself via FluxCD.
- That the platform's desired state is defined by the Platform Package, and FluxCD takes the charts from an OCI registry.
- The two FluxCD objects: `HelmRepository` — the source of charts, `HelmRelease` — the statement about an installed chart.
- That a chart is the unit of packaging, and every item in the managed applications catalog is a chart.
- That Cozystack itself is installed with the `cozy-installer` Helm chart.
- The chain: object → `HelmRelease` → chart → operator → pods.
- What `Namespace`, `Pod`, `Deployment`, `ReplicaSet`, `Service`, `PVC` and `Secret` are — one sentence for each.
- The three `Service` types: `ClusterIP`, `NodePort`, `LoadBalancer`.
- Commands: `kubectl get pods -A`, `kubectl describe pod` for events, `kubectl api-resources`, `--dry-run=server`, `helm upgrade --install`.

Further reading: [platform overview](#) · [FluxCD concepts](#)

8. Cheat sheet

One page for the whole exam: what is responsible for what, what it gets swapped for in the answer options, and how to prepare day by day.

The last page before the exam. Reading it instead of the lessons is pointless — it does not explain, it reminds. But skimming it half an hour before your attempt is exactly right.

What is responsible for what

The third column matters more than the first two: the exam builds wrong answer options out of substitutions, and almost all of them come from this list.

Task	Component	What it gets swapped for in the options
Node operating system	Talos Linux	Ubuntu, CoreOS
Virtual machines	KubeVirt	Proxmox, oVirt
Control plane of tenant clusters	Kamaji	Cluster API on its own
Block storage	LINSTOR / DRBD	Ceph, Longhorn
Object storage	SeaweedFS	MinIO, Ceph RGW
Pod network, policies	Cilium	Calico, Flannel
Tenant networks, VPC	Kube-OVN	Cilium, Calico
External addresses	MetalLB	cloud load balancer
Metrics	VictoriaMetrics	Prometheus
Logs	VictoriaLogs	Loki , Elasticsearch
Dashboards	Grafana	Kibana

Task	Component	What it gets swapped for in the options
Alerts	VMAAlert → Alerta	Alertmanager
Configuration delivery	FluxCD	ArgoCD
Sign-in	Keycloak	Dex
External DNS records	ExternalDNS	CoreDNS
Certificates	cert-manager	ExternalDNS

In bold — the two most common traps. Prometheus and Loki are not in the platform at all.

Numbers that get asked

What	Value
Minimum nodes	3
Per node	8 cores, 24 GB of memory
Disks per node	50 GB + 256 GB
Latency between nodes	under 10 ms
Network	a single L2 segment
cpuAllocationRatio	10 by default
Namespace name length	up to 63 characters

Names to know verbatim

Installation variants: `isp-full`, `isp-full-generic`, `isp-hosted`, `default`. The old names `paas-full` and `distro-full` were replaced in version 1.5 — in the answer options they are a trap.

Tenant switches: `etcd`, `monitoring`, `ingress`, `seaweedfs`.

Access labels: `policy.cozystack.io/allow-to-apiserver`,
`policy.cozystack.io/allow-to-etcd`. The value is the string `"true"`.

Backup objects: `BackupClass`, `Plan`, `BackupJob`, `Backup`, `RestoreJob`. The ready-made class is `cozy-default`.

Platform API group: `apps.cozystack.io/v1alpha1`. It is served by the **aggregated API server** `cozystack-api` in `cozy-system` — not a CRD. That is why `kubectl get tenants` works.

Manifest fields: `apiVersion` — the API version, `kind` — the type, `metadata` — name and labels, `spec` — **the desired state**, `status` — the actual state, written by the controller.

A tenant's kubeconfig is assembled from a **token**, a **CA certificate** and the **server address**: the first two come from the tenant's secret of the same name, the address from the administrator's kubeconfig.

suspend on a HelmRelease — the controller stops reconciling; pods keep running, manual changes are kept until reconciliation is turned back on.

Chains

Layers: `Talos` → `Kubernetes` → `Cozystack`

Order: `object` → `HelmRelease` → `chart` → `operator` → `pods`

Metric: `pod` → `vmagent` → `VictoriaMetrics` → `Grafana`

Log: `pod` → `fluent-bit` → `VictoriaLogs` → `Grafana`

Alert: `VMAlert` → `Alerta` → email and messengers

Namespace name: `tenant-` + chain of ancestors without the root

What exists and what does not

Exists: live migration of machines, snapshots, GPU passthrough, scheduled backups, separate tenant networks.

Does not exist: automatic balancing across nodes, like DRS. Incremental backups of virtual machines. The ability to disable tenant isolation. **Automatic restart of a virtual machine on another node after a failure** — that requires fencing, which is not included.

Changed in 1.5: node preparation for GPU passthrough has been automated — node labeling and the list of allowed devices. Passthrough itself existed before.

Five-day preparation plan

Day	What to do
1	Lesson 7 — it is about the ideas everything rests on. Then lesson 1
2	Lesson 2 — the densest one. Go over whatever you missed on day one
3	Lessons 3 and 4
4	Lessons 5 and 6
5	Practice questions several times, catch up on weak topics, this cheat sheet

If you do not have five days — read straight through in an evening, run the practice questions until you stop making mistakes, and go take the exam.

How the exam works

60 questions, 90 minutes. In English — if it is not your native language, request an extra 30 minutes in advance. Some questions have several correct answers, and they count only if answered in full. The result is pass or fail, with an overall score and a breakdown by topic. Two attempts, one week apart.

The passing score is not published. Prepare to know the material, not to hit a number.